Week 12: Capstone Project - Lab Guide

HPC & Cluster Computing

Objective: Verify the complete functionality of your Cluster. **Goal**: Pass the "Final Acceptance Test".

---

1. Infrastructure Verification

Ensure that your Proxmox cluster is fully operational. Log into your Master Node (`hpc-master`) via SSH to begin your work.

- [ ] **Nodes Alive**: `hpc-master`, `compute-01`, and `compute-02` are powered on in Proxmox.
- [ ] **Network**: Master can ping the compute nodes over the isolated network: `bash ping -c 1 10.0.0.101 # compute-01 ping -c 1 10.0.0.102 # compute-02`
- [ ] **DNS**: Configure `/etc/hosts` on the Master so you can use names (`compute-01`, `compute-02`).
- [ ] **OpenHPC**: Verify repositories are enabled on the master: `bash dnf repolist | grep OpenHPC`

---

2. Warewulf Verification (Provisioning)

Demonstrate that your cluster is aware of its nodes.

- [ ] **Command**:

```
bash sudo wwctl node list
```

- [ ] **Requirement**: All nodes must show status `READY`/`SEEN`.

---

3. Monitoring Verification (Grafana)

To keep the setup light, we will run the monitoring stack in **Containers** using Podman.

3.1 Deploy Prometheus Container

- [ ] **Create Config**: ```bash cat < prometheus.yml global: scrape_interval: 15s scrape_configs:
    - job_name: 'cluster' static_configs:
        - targets: ['10.0.0.101:9100', '10.0.0.102:9100'] EOF ```
- [ ] **Run Container**: `bash podman run -d --name prometheus \ -p 9090:9090 \ -v ./prometheus.yml:/etc/prometheus/prometheus.yml \ prom/prometheus`

3.2 Deploy Grafana Container

- [ ] **Run Container**: `bash podman run -d --name grafana -p 3000:3000 grafana/grafana`

3.3 Verification

- [ ] **Action**: Open `http://10.0.0.1:3000` (Grafana) in your browser.
- [ ] **Requirement**: Add Prometheus (`http://10.0.0.1:9090`) as a Data Source and verify connection.

---

4. Storage Checklist (Data)

- [ ] **Home (NFS)**:
  - Create `~/test_nfs` on Master.
  - Verify it appears on `compute-01` and `compute-02` immediately.
  - *Note: For this Capstone, we rely entirely on NFS shared home directories.*

---

5. The Parallel Programming Challenge

Instead of running pre-built software, you must demonstrate your ability to write and run parallel code. You will implement **Parallel Matrix Multiplication** using MPI.

5.1 The Code (`matrix_mult.py`)

Create this Python script in your home directory (which is shared via NFS).

- [ ] **Create Script**: `bash nano matrix_mult.py`
- [ ] **Paste Content**: ```python from mpi4py import MPI import numpy as np import time

  comm = MPI.COMM_WORLD rank = comm.Get_rank() size = comm.Get_size()

  N = 1000 # Matrix Size (1000x1000)

  if rank == 0: print(f"Master: Initializing matrices size {N}x{N}...") A = np.random.rand(N, N) B = np.random.rand(N, N) start_time = time.time() else: A = None B = None

  # Broadcast B to everyone (Not memory efficient, but simple)

  B = comm.bcast(B, root=0)

Scatter A (Distribute rows)

Each process gets N/size rows

local_rows = N // size local_A = np.zeros((local_rows, N)) comm.Scatter(A, local_A, root=0)

Compute Local Sub-Matrix

if rank == 0: print(f"Computing on {size} processors...")

local_C = np.dot(local_A, B)

Gather results back to Master

if rank == 0: C = np.zeros((N, N)) else: C = None

comm.Gather(local_C, C, root=0)

if rank == 0: end_time = time.time() print(f"Done! Execution Time: {end_time - start_time:.4f} seconds.") # Optional: Verify with serial calculation for small N ```

5.2 The Job Script (`submit_matrix.sh`)

You must use the Scheduler to run this.

- [ ] **Create Script**: `bash nano submit_matrix.sh`
- [ ] **Paste Content**: ```bash #!/bin/bash #SBATCH --job-name=MATRIX_MULT #SBATCH --nodes=2 #SBATCH --ntasks=2 # 1 Task per Node #SBATCH --

partition=debug #SBATCH --output=matrix_%j.log #SBATCH --chdir=/home/openeuler # Ensure we run in the shared home

echo "Running on: $SLURM_JOB_NODELIST"

Load Modules (Ensure your environment has mpi4py)

If using system python:

---

dnf install python3-mpi4py mpich -y (inside image)

---

module load mpi

mpirun -np 2 python3 matrix_mult.py ```

5.3 Submit and Verify

- [ ] **Submit**: `bash sbatch submit_matrix.sh`
- [ ] **Check Output**: `bash cat matrix_*.log` *Did it print the Execution Time?*

---

6. Submission

Prepare a presentation or report containing screenshots of:

1. **Warewulf**: Output of `wwctl node list`.
2. **Monitoring**: Your Grafana Dashboard active.
3. **Result**: The log file from your Matrix Multiplication job showing success.

Course Index